Computational geometry

How can exact geometric predicates support efficient and correct algorithms?

We study **orientation → segment intersection and sweep → convex hull → closest pair**. Prerequisites include sorting, balanced search trees, divide and conquer, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From geometry to algorithms

Problem	Focus
Orientation and intersection	Determinant signs, boundaries, and numeric precision.
Sweep line	Event order, status, and adjacency.
Convex hull	Sorting, stack invariants, and degeneracy.
Closest pair	Divide and conquer, the strip, and geometric packing.

Each result depends on explicit input assumptions and a correctness proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Answer geometry questions with local signs

The input consists of planar points and closed segments. Fix coordinates and intersection semantics first. Use orientation to test a pair of segments, then use a sweep to decide whether any intersecting pair exists.

All numerical examples use exact integers or rationals. Time bounds initially count each exact arithmetic operation as constant; large-integer bit costs are additional. Sources are CLRS 33.1–33.2 and MIT 2004 L17. Original notebook PDF43–47 supplies the geometry outline and turn diagrams.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A positive left turn depends on operand order

Use an x-axis pointing right and a y-axis pointing up.

$$\operatorname{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

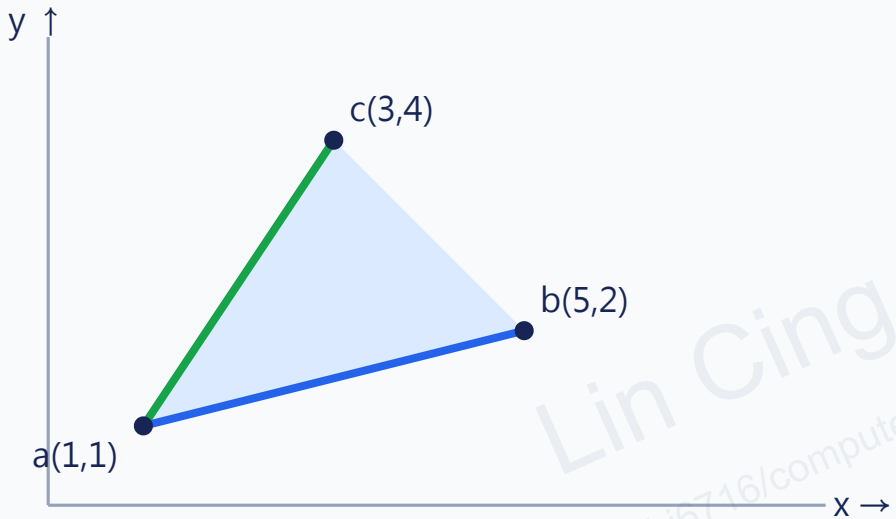
A positive value means a left turn along a,b,c; a negative value means a right turn; zero means collinearity. Its magnitude is the parallelogram area. The triangle area is half that magnitude.

CLRS `DIRECTION(a,b,c)` uses $(c - a) \times (b - a)$, the negative of our formula. Reverse its inequalities when translating code. The SVG rendering maps mathematical y upward into screen coordinates; it does not change our sign convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A positive left turn depends on operand order (continued)

Orientation signs and coordinates



$(b-a) \times (c-a)$

$4 \cdot 3 - 1 \cdot 2 = +10$

Left turn; triangle area 5

DIRECTION = −10

Mathematical y points up

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Evaluate one determinant

Let a=(1,1), b=(5,2), and c=(3,4). The displacement vectors are b−a=(4,1) and c−a=(2,3).

$$\operatorname{orient}(a,b,c) = 4 \cdot 3 - 1 \cdot 2 = 10 > 0.$$

Thus c lies left of the directed line ab, and the triangle area is 5. Exchanging b,c gives −10. Translating all points preserves the displacement vectors. If c=(9,3), its displacement is (8,2), so the determinant is 4 • 2−1 • 8=0. This proves collinearity, but does not prove that c belongs to closed segment ab.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closed intersection and proper crossing

The closed segment $ab = \{a + t(b - a) : 0 \le t \le 1\}$ contains its endpoints. We also allow a=b, a point segment.

Here a proper crossing means one interior intersection of two noncollinear segments. If a specification uses “strict intersection” to include collinear interior overlap, define that separately.

Geometric case	Closed intersection	Proper crossing
Two interiors cross	Yes	Yes
Endpoint or T contact	Yes	No
Collinear overlap	Yes	No
Point segment on another segment	Yes	No
Only extensions meet	No	No

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Closed intersection and proper crossing (continued)

Boundary cases for closed segments



Proper crossing | Endpoint contact | Collinear overlap | Point segment

Blue/red are inputs; green is overlap; gold marks contact.

All four intersect as closed segments; only the first is proper.

Endpoints, collinearity, and point segments need explicit semantics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

on needs collinearity and both coordinate bounds

```
on(a,b,p):
    return orient(a,b,p) == 0 and
           min(ax,bx) <= px <= max(ax,bx) and
           min(ay,by) <= py <= max(ay,by)
```

With a=(2,1), b=(2,5), p=(2,8), collinearity and the x bound hold, but p is outside ab. The y bound is necessary. When a=b, the two coordinate bounds require p=a, so point segments work too.

CLRS ON-SEGMENT assumes its caller has established collinearity. Our on includes that condition. A bounding box alone cannot establish segment membership.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Four orientations complete the test

Write o1=orient(a,b,c), o2=orient(a,b,d), o3=orient(c,d,a), and o4=orient(c,d,b).

```
opposite(u,v): return (u<0 and v>0) or (u>0 and v<0)
intersect(a,b,c,d):
    compute o1,o2,o3,o4
    if opposite(o1,o2) and opposite(o3,o4): return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

The first branch is exactly proper crossing. The second handles endpoints, collinear overlap, and point segments. Compare signs rather than multiplying o1 · o2, which would add another overflow risk. Vertical segments need no slope calculation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both straddling conditions are necessary

Take ab from (0,0) to (6,0), and cd from (7,−1) to (7,2).

$$(o_1, o_2, o_3, o_4) = (-6, 12, 21, 3).$$

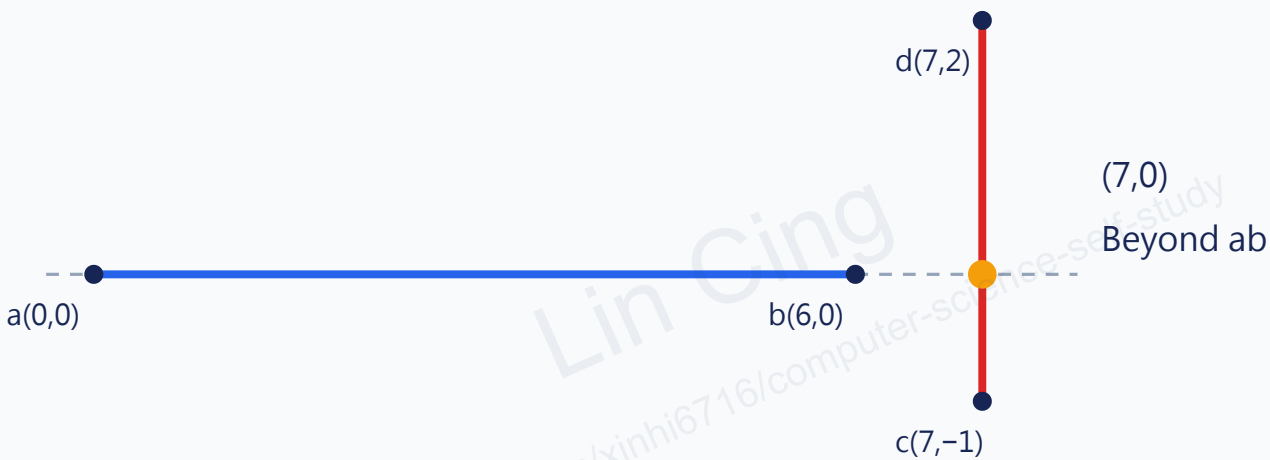
The endpoints c,d straddle the line through ab, but a,b lie on the same side of the line through cd. The line intersection (7,0) is beyond ab, so the segments are disjoint.

Without zero orientations, the first sign pair puts the line intersection inside cd; the second puts that same point inside ab. Both are necessary. For collinear closed intervals, overlap implies that an endpoint of one belongs to the other. The four on tests therefore cover the remaining cases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both straddling conditions are necessary (continued)

Straddling an extension is insufficient



o1=−6, o2=12; o3=21, o4=3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact signs need enough bits

If each coordinate has absolute value at most B, differences are at most 2B. A conservative bound is $|\operatorname{orient}| \leq 8B^2$. Widen coordinates before subtraction and multiplication; casting the result is too late.

For a=(0,0), b=(100000,0), c=(0,100000), orientation is 10^{10}, beyond signed 32-bit range. Full signed 32-bit coordinates also do not guarantee safe signed 64-bit intermediate results.

Floating subtraction can lose significant digits. Scaling coordinates by λ scales the determinant by λ^2, so a fixed epsilon has no scale-independent guarantee. Treating values within epsilon as equal can also violate comparator transitivity. Use exact arithmetic or error-bounded filters with increased precision when inconclusive. An arbitrary epsilon is not a proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Replace all pairs with events

Testing every pair of n segments uses $\Theta(n^2)$ predicates. Detection needs one witness. Reporting must output all k intersecting pairs. Since k can be $\Theta(n^2)$, the $O(n \log n)$ detection bound cannot describe reporting all intersections.

A vertical sweep uses x as time. Its event schedule stores endpoints. Its status T stores segments crossing the sweep line, ordered bottom to top.

Our initial simplified version assumes nonvertical, nonzero segments, distinct endpoint x-coordinates, no three segments through one point, and no collinear overlap. The pair predicate has none of these restrictions. They are assumptions for the sweep status and proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both insertion and deletion create neighbors

```
T = empty balanced search tree
sort the 2n endpoints by x
for endpoint p:
    if p is the left endpoint of s:
        insert s in T
        test s against predecessor and successor; if any intersects: return true
    else:
        test predecessor(s) against successor(s); if they intersect: return true
        delete s from T
return false
```

Each predicate tests the complete segments, not only the current sweep position. Detection can therefore return before the actual crossing. At deletion, test the two segments formerly separated by s. Omitting this test can miss a newly adjacent pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace numerical coordinates at every event

a:(0,0)→(10,5), c:(1,2)→(3,2), b:(2,6)→(9,0). Status is listed bottom to top.

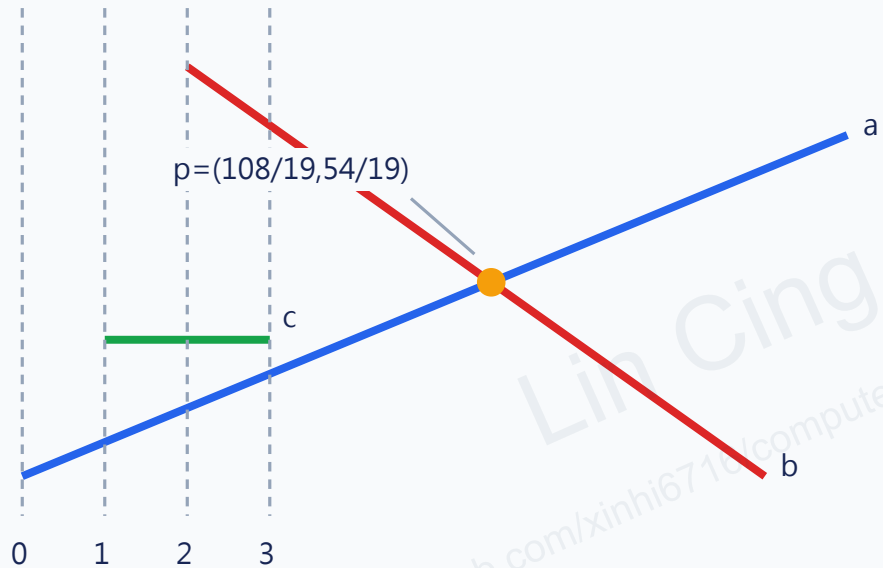
Event	Heights and status	New tests
x=0 insert a	[a]	None
x=1 insert c	a=1/2,c=2; [a,c]	a,c disjoint
x=2 insert b	a=1,c=2,b=6; [a,c,b]	c,b disjoint
x=3 before deleting c	a=3/2,c=2,b=36/7; [a,c,b]	a,b intersect; return

Segment a reaches y=2 at x=4, beyond c. Segment b reaches y=2 at x=20/3, also beyond c. Solving $x/2 = (54 - 6x)/7$ gives the a,b intersection $(108/19, 54/19)$. Deleting c occurs before this crossing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace numerical coordinates at every event (continued)

Deleting c detects a future intersection



Event x	T bottom→top
0	[a]
1	[a,c]
2	[a,c,b]
3	a,b → true

x=3 < 108/19

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why neighbor tests decide existence

Suppose the detector has not returned. Consider segments a,b at the leftmost intersection p. No crossing precedes p, so the status order cannot change on its own to the left of p. With no triple intersection, a,b must become adjacent before or at p.

Their first adjacency arises either when one is inserted or when the last intervening segment is deleted. The algorithm tests complete segments at both kinds of event, so it returns before or at p. If it returned earlier on another pair, the existence answer is already correct.

A balanced tree performs a constant number of O(log n) operations per event. There are 2n endpoints, sorting costs O(n log n), and total space is O(n). The proof relies on stopping after one witness; it does not prove reporting correctness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal x-coordinates need an event policy

To allow equal endpoint x-coordinates in the nonvertical version, sort by (x,e,y,id), with e=0 for a left endpoint and e=1 for a right endpoint. Process all starts before ends at the same x, and increasing y within each kind. The id makes identical events reproducible.

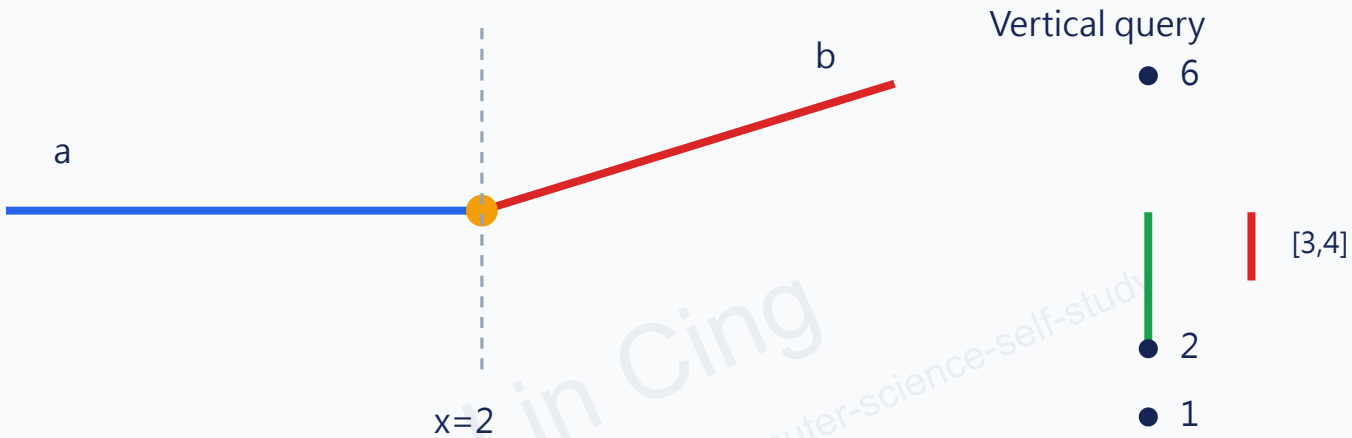
a:(0,1)→(2,1) and b:(2,1)→(4,2) meet at their shared endpoint. Deleting a before inserting b leaves them absent from T at different times and misses their contact. Inserting b first allows the test.

Equal height can mean an actual intersection; an arbitrary id is not a substitute for testing it. Multiple segments at one point require grouped events and equal-height checks, or the simplifying assumption must remain. CLRS 33.2 excludes triple intersections in its initial proof.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Equal x-coordinates need an event policy (continued)

Same-x ordering and closed intervals



Insert b before deleting a to detect endpoint contact.

[2,4] hits y=2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A vertical segment is an interval query

A vertical segment v:(x,ylo)→(x,yhi) cannot use one ordinary height key. It may meet any active nonvertical segment in that interval.

At the same x, insert nonvertical starts, query closed vertical intervals against status heights, then delete nonvertical ends. For detection, find the first height $\geq$ylo and test whether it is $\leq$yhi. Sort vertical intervals at that x by ylo to check their overlap separately. A zero-length segment is a point interval. Shared nonvertical endpoints also need checking.

At x=2 with active heights 1,2,6, interval [3,4] has no hit; [2,4] hits the boundary at y=2. These are required operations for a generalized implementation. The earlier simplified pseudocode does not implement all grouped-event and overlap cases, so it does not claim universal degeneracy support.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Printing and continuing breaks the status

a:(0,0)→(4,4), b:(0,4)→(4,0), c:(0,5)→(4,5). Before x=2 the order is [a,b,c]; after x=2 it is [b,a,c].

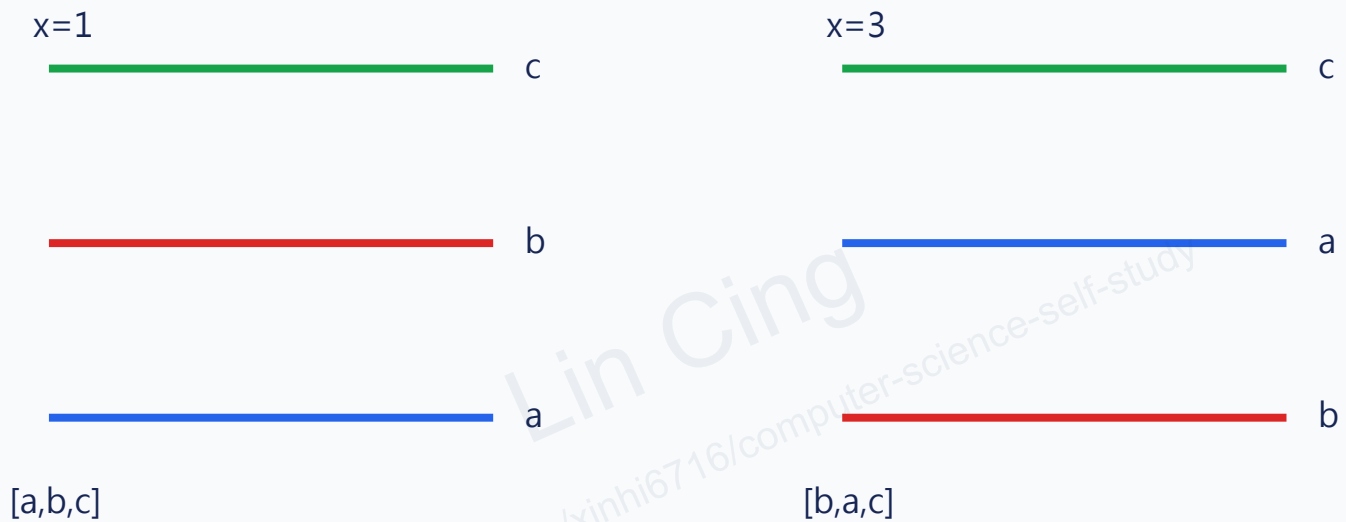
If insertion discovers a,b and merely prints them, an endpoint-only algorithm does not swap their order between endpoint events. Its tree stops representing geometric neighbors. The first tested intersection also need not be the leftmost one, since predicates test entire segments.

The MIT04 reporting version additionally schedules predicted intersection events, swaps neighbors at each crossing, and updates events with duplicate and stale-event handling. In general position its time is O((n+k) log n). With collinear overlap, define whether the output consists of intersecting pairs, points, or overlapping segments; there may be infinitely many shared points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Printing and continuing breaks the status (continued)

Geometric order changes after a crossing



An intersection event must exchange order

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Align predicate semantics and data structures

Layer	Required specification	Checked example
Orientation	Axes, operand order, arithmetic	Determinants 10 and −10
Pair intersection	Closed or proper, point segments	Overlap, endpoints, extensions
Sweep detection	Events, neighbors, stopping rule	Deleting c exposes a,b
Generalization	Shared points, verticals, collinearity	Insert, query, then delete
Reporting	Intersection events and output type	Update order after a crossing

Exercises A10-P01–P06 check numerical signs, boundary cases, numerical reliability, event traces, tied coordinates, and detection versus reporting. Orientation will also determine which vertices a convex-hull scan keeps or pops.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hull vertices and closest pairs need different certificates

A hull scan removes points that cannot remain extreme vertices. Closest pair removes pairs that cannot improve a distance bound. Both reductions require geometric invariants. The fixed inputs and executable checks are in data.json and check.py.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Specify the output policy

Return extreme hull vertices counterclockwise, without repeating the first vertex. Omit intermediate collinear boundary points. Return endpoints for an all-collinear set, the point for a singleton, and an empty list for an empty input. Deduplicate hull coordinates. Keep distinct point ids for closest pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Use one fixed hull input

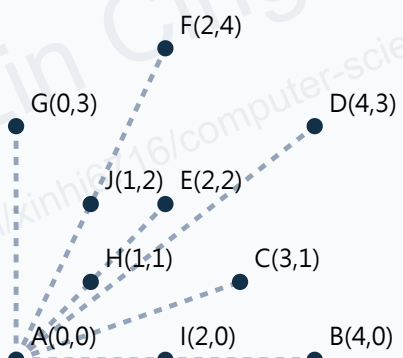
A(0,0), B(4,0), C(3,1), D(4,3), E(2,2), F(2,4), G(0,3), H(1,1), I(2,0), J(1,2). A has minimum y, breaking ties by minimum x. orient(a,b,c)=(b−a)×(c−a) is positive for a left turn.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resolve equal polar angles

Raw polar order about A is I,B,C,D,H,E,J,F,G. Keep only the farthest point on each ray. This removes I, H, and J, leaving B,C,D,E,F,G. Exact comparisons use cross products and squared distances rather than equality of floating atan2 values.

Polar order about A; remove I, H, J



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pop on every nonleft turn

For each point p in polar order, while $|S| \geq 2$ and orient(S[-2],S[-1],p)≤ 0, pop. Then push p. The nonstrict test implements the extreme-vertex policy. Changing it to <0 alone does not provide a complete boundary-point policy; angle ties and the closing edge also need attention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Replay every stack update

Point	Popped	Stack bottom to top
B	-	A,B
C	-	A,B,C
D	C	A,B,D
E	-	A,B,D,E
F	E	A,B,D,F
G	-	A,B,D,F,G

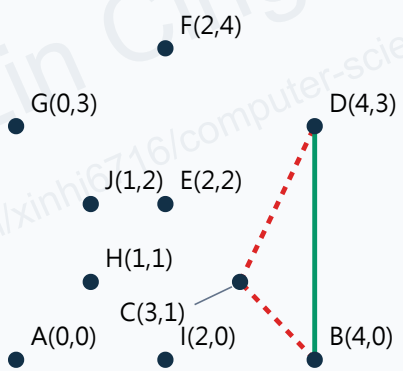
Start with S=[A]. Each row lists all pops for that insertion. On this nondegenerate input this equals the standard version initialized with the first three retained points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Explain why a pop is safe

At D, orient(B,C,D)=−3, so pop C and obtain A,B,D. At F, orient(D,E,F)=−4, so pop E and obtain A,B,D,F. At G no point is popped. A popped point lies inside or on the triangle formed by A, its retained predecessor, and the incoming point, and is not an extreme vertex of the processed prefix.

Read D, pop C, retain A B D



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

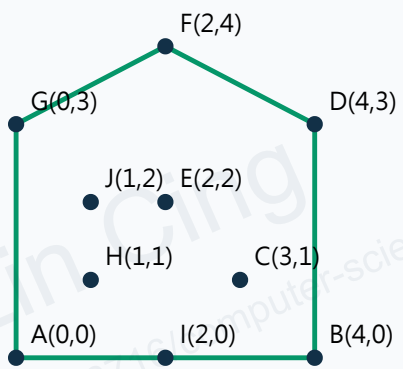
Prove the prefix invariant

After each insertion, S contains exactly the extreme vertices of the processed prefix together with A, in counterclockwise order. Initialization holds. Pops preserve its convex hull; the next polar point extends its outside chain. At termination every input point has been considered. The answer is A,B,D,F,G, and every input point lies on or to the left of each directed hull edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove the prefix invariant (continued)

Extreme hull vertices A B D F G



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare Graham scan with Jarvis march

Graham uses $O(n \log n)$ sorting and $O(n)$ scanning because each retained point is pushed once and popped at most once. Space is $O(n)$. Jarvis scans all n points for each next supporting edge and takes $O(nh)$ for h hull vertices. Choose the farthest collinear candidate and stop on returning to the start. If $h=n$, its cost is $O(n^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fix the closest-pair input

A(−4,0), B(−3,4), C(−1,1), D(0,5), E(0,1.5), F(2,3), G(4,0), H(5,5). Compare squared distances and take a square root only for presentation. Distinct ids at the same coordinates immediately certify a distance of zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

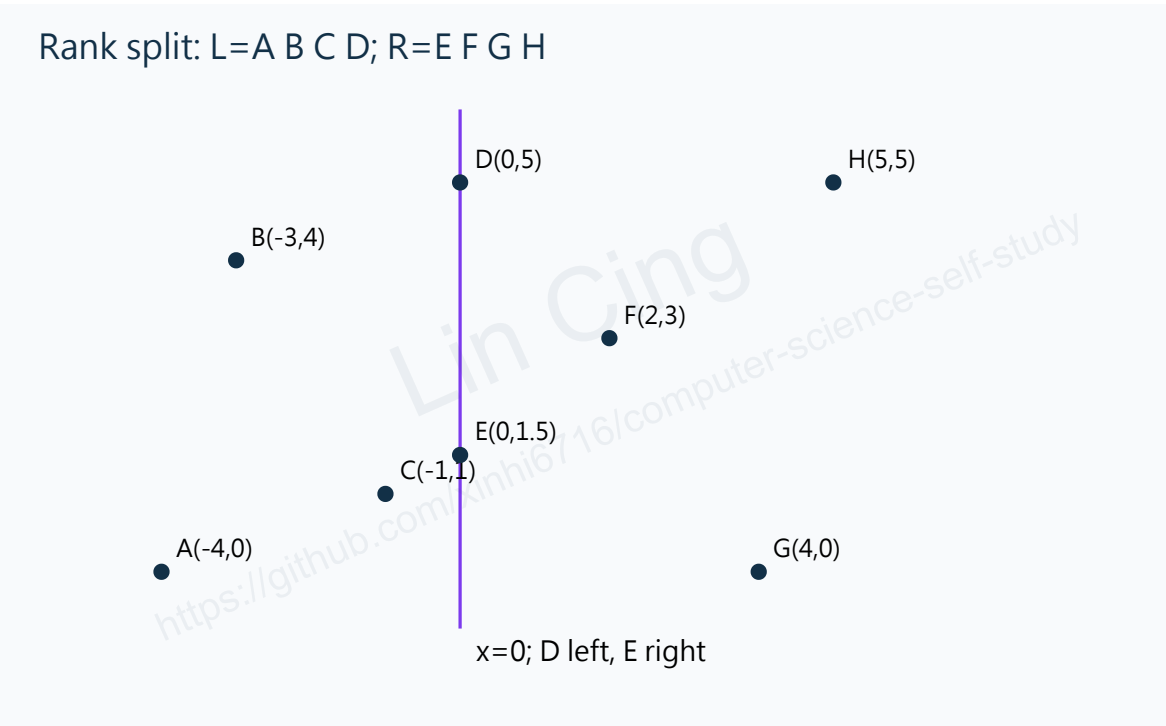
Presort both coordinate orders

Sort X by (x,id) and Y by (y,id) once. Each recursive call receives the same ids in both arrays. Split X by rank, construct a set of left ids, and stably filter Y by id membership. Do not sort Y again at every recursive level.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partition repeated x values by identity

X=A,B,C,D,E,F,G,H. L=A,B,C,D and R=E,F,G,H with dividing line x=0. D and E share x=0 but belong to different subproblems. Assigning every $x \leq 0$ point to the left can destroy balance.



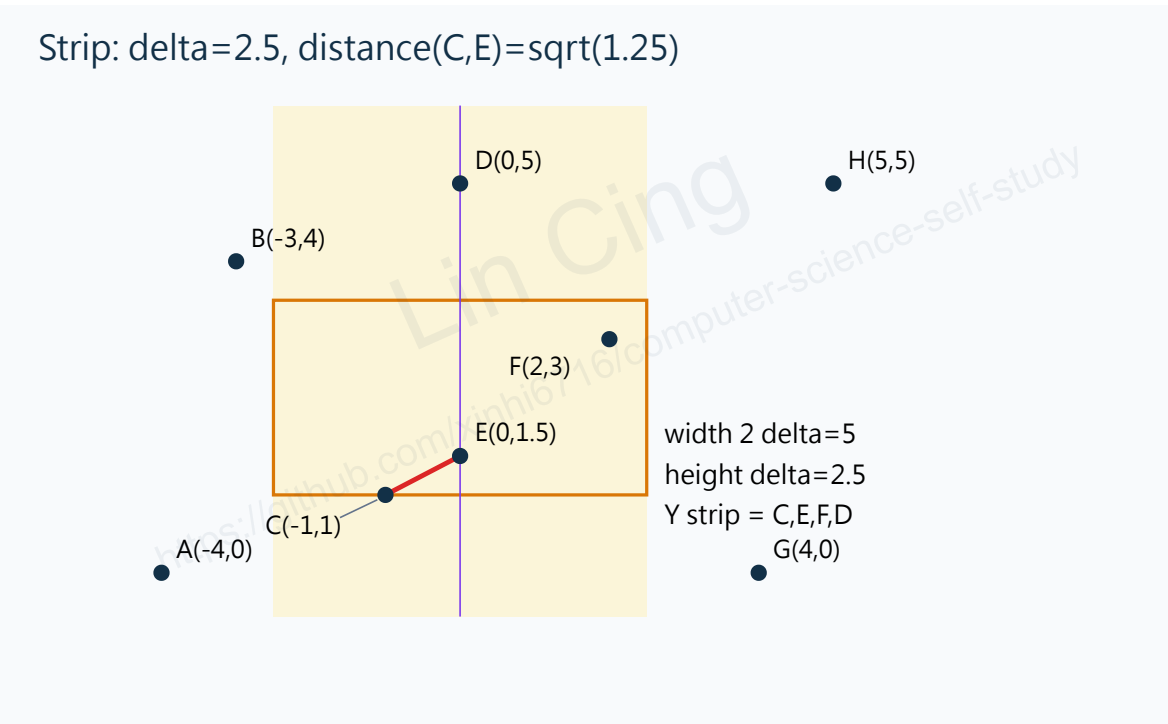
Separate recursive and crossing answers

Brute-force subproblems of size at most 3. The minimum left squared distance is 10, attained by A-C and B-D. On the right it is 6.25 for E-F. Thus $\delta = \min(\sqrt{10}, 2.5) = 2.5$. Any improving crossing pair has both $|x - 0| < \delta$ and vertical separation $< \delta$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scan the strip in y order

Stably filter Y by |x−line|<δ. The strip order is C(−1,1), E(0,1.5), F(2,3), D(0,5). C-E has squared distance 1.25 and distance about 1.118. Other pairs passing the original vertical test are C-F with 13, E-F with 6.25, and F-D with 8. Only C-E improves δ.



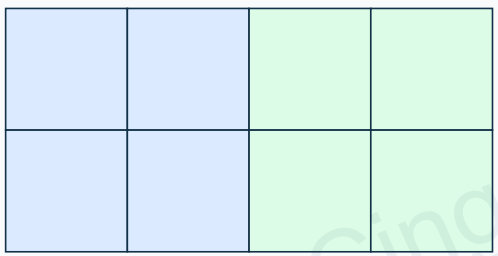
Derive the packing bound

Assume $\delta > 0$ and that each recursive side has pairwise distances at least δ. The later points within vertical distance δ of p lie in a 2δ-wide, δ-high rectangle. Split each side into four $\delta/2$ by $\delta/2$ cells. A cell holds at most one point of that side since its diagonal is $\delta/\sqrt{2} < \delta$. Points on the dividing line retain their assigned side.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derive the packing bound (continued)

Four delta/2 cells per side



2 delta by delta; cell diagonal = delta/sqrt(2)

At most 4 left + 4 right = 8 including p

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Seven successors has explicit assumptions

There are at most four left points and four right points in the rectangle, including p. At most seven points follow p. Y uses a consistent (y,id) order and a half-open cell ownership rule resolves boundaries. The proof requires separation within each side, not between the two sides. If δ=0, return before using the positive-width packing argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Keep the combine step linear

Presorting takes $O(n \log n)$. Splitting X, filtering Y, building the strip, and making constantly many comparisons per point take $O(n)$ per call. $T(n)=2T(n/2)+O(n)=O(n \log n)$. Sorting strips in each call instead gives $O(n \log^2 n)$. Depth is $O(\log n)$; sequential recursion with released temporary arrays needs $O(n)$ extra space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Read sources and check the actual input

CLRS third edition §§33.3–33.4, printed pages 1029–1043, PDF pages 1050–1064. Original scanned note PDF pages 46–47 support Graham scan and hull applications. MIT 2004 L18 PDF pages 3–11 cover closest pair and packing. The local 2005 inventory contains no geometry lecture. check.py enumerates all 45 hull-input and 28 closest-input pairs and checks support edges, the stack trace, and the divide-and-conquer result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 33. MIT 6.046J Fall 2004 computational geometry lecture slides, segment intersection and closest pair. Original handwritten algorithms notes (演算法_wjungle), computational geometry pages.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study